4회차 과제 — 전처리와 저장

1. 처리 흐름 — Kafka 에 쌓인 메시지를 전처리 후 2곳에 저장한다

Kafka user-events → Spark Structured Streaming → ① Parquet (전체 보관) + ② PostgreSQL (최근 30일, 조회용)
· 코드: preprocess.py

순서	하는 일	결과
① 읽기	Kafka 토픽을 묶음(micro-batch)으로 읽음. 읽은 위치를 체크포인트에 기록	다시 실행하면 그다음부터 읽음
② 파싱	JSON 을 정해 둔 스키마로 풀고 타입 변환	시각 → timestamp, ID → 숫자
③ 검증	이상한 메시지에 실패 이유를 붙임	이유 있으면 DLQ 토픽으로
④ 중복 제거	같은 event_id 는 하나만	원본 중복 116건 제거
⑤ Parquet 저장	날짜별 폴더에 추가	지우지 않음
⑥ PostgreSQL 저장	임시 테이블 → events 로 옮기며 이미 있는 건 건너뜀	다시 돌려도 중복 없음
⑦ 오래된 행 삭제	가장 최근 이벤트 기준 30일 이전 삭제	PostgreSQL 은 최근 30일만

① 읽기 — Kafka 에서 묶음으로, 읽은 위치는 체크포인트에

preprocess.py 285~300줄

```
stream = (spark.readStream.format("kafka")
          .option("subscribe", "user-events")
          .option("startingOffsets", "earliest")
          .load())


writer = (stream.writeStream
          .foreachBatch(make_batch_handler(args))
          .option("checkpointLocation", "checkpoint/preprocess"))
query = writer.trigger(availableNow=True).start()
```

Spark 가 Kafka 토픽을 구독해서 묶음 단위로 읽는다. **checkpointLocation** 에 어디까지 읽었는지(offset) 기록되므로 다시 실행하면 그다음부터 읽는다. 묶음 하나가 들어올 때마다 **foreachBatch** 에 넘긴 함수가 ②~⑦ 을 처리한다.

② 파싱 — JSON 을 풀고 타입을 맞춘다

preprocess.py 86~93줄

```
.withColumn("m", F.from_json("raw_value", MESSAGE_SCHEMA))
.withColumn("event_time", F.to_timestamp("m.event_time"))
.withColumn("visitor_id", F.col("m.payload.visitor_id"))   # LongType in schema
```

Kafka 메시지는 바이트라 문자열로 바꾼 뒤, 미리 정해 둔 스키마(MESSAGE_SCHEMA)로 JSON 을 푼다. 시각 글자는 **to_timestamp** 로 시각 타입이 되고, ID 는 스키마에서 숫자 타입으로 읽힌다. 못 읽으면 null 이 된다.

③ 검증 — 실패 이유를 붙이고, 이유가 있으면 DLQ 로

preprocess.py 105~110줄, 234줄

```
reason = (F.when(all_null, "invalid_json")
           .when(F.col("event_id").isNull(), "missing_event_id")
           .when(~F.col("event_type").isin("view", "addtocart", "transaction"), "bad_event_type")
           .when(F.col("event_time").isNull(), "bad_event_time")
           .when((F.col("event_type") == "transaction")
                 & F.col("transaction_id").isNull(), "transaction_without_id"))


bad.write.format("kafka").option("topic", "user-events-dlq").save()
```

메시지마다 실패 이유를 계산한다. 이유가 없으면 정상, 있으면 원문과 이유를 같이 **user-events-dlq** 토픽으로 보낸다. 버리지도 멈추지도 않고 격리해서 나중에 보고 고친다. (테스트: JSON 아님 → invalid_json, event_type=purchase → bad_event_type)

④ 중복 제거 + ⑤ Parquet 저장

preprocess.py 241~245줄

```
dedup = (good.dropDuplicates(["event_id"])
             .withColumn("dt", F.to_date("event_time")))
dedup.write.mode("append").partitionBy("dt").parquet("data/parquet/events")
```

같은 event_id 는 하나만 남긴다. 이벤트 날짜로 **dt** 칸을 만들어 날짜별 폴더에 Parquet 로 추가한다. 실행하면 **data/parquet/events/dt=2015-05-03/** 같은 폴더가 생긴다. 지우지 않는 전체 보관본이다.

⑥ PostgreSQL 저장 — 임시 테이블을 거쳐 이미 있는 건 건너뛴다

preprocess.py 190줄, 126~131줄

```
good.write.format("jdbc").option("dbtable", "events_staging").mode("overwrite").save()


-- SQL
INSERT INTO events (...)
SELECT ... FROM events_staging
ON CONFLICT (event_id) DO NOTHING;
```

Spark 에서 바로 events 에 넣으면 같은 event_id 가 왔을 때 에러가 난다. 그래서 임시 테이블(**events_staging**)에 통째로 넣고, SQL 로 events 에 옮기면서 이미 있는 event_id 는 건너뛴다(**ON CONFLICT DO NOTHING**). 몇 번을 다시 돌려도 중복이 안 생긴다.

⑦ 오래된 행 삭제 — PostgreSQL 은 최근 30일만

preprocess.py 173~174줄

```
DELETE FROM events
WHERE event_time < (SELECT max(event_time) FROM events) - interval '30 days';
```

가장 최근 이벤트 시각 기준 30일 이전 행을 PostgreSQL 에서 지운다. 2015년 데이터를 리플레이하는 중이라 now() 가 아니라 데이터 안의 최신 시각을 기준으로 한다. Parquet 은 건드리지 않는다. 기간은 **--pg-retention-days** 로 바꿀 수 있다.

실행 결과 — 45일치

```
INFO batch=0 kafka=919497 parsed_ok=919497 dlq=0 dedup=919381(-116)
     | pg inserted=919381 skipped_existing=0
     | retention(30d) deleted events=301050 | 62.5s
```

단계	건수
Kafka 에서 읽음	919,497
DLQ 로 감	0
중복 제거 후	919,381 (116건 제거)
Parquet 저장	919,381 · 2015-05-03 ~ 06-17 (46일)
PostgreSQL 에 남음	618,331 · 2015-05-18 ~ 06-17 (31일)
30일 밖이라 삭제	301,050
걸린 시간	62.5초

2. 입력 · 출력 예시 — 원본 CSV 한 줄과 전처리 후 저장된 한 줄

입력: 원본 events.csv 한 줄 (2015-06-10 구매 이벤트)

```
timestamp,visitorid,event,itemid,transactionid
1433894433399,569031,transaction,417927,1189
```

출력: 전처리 후 저장된 한 줄 (Parquet · PostgreSQL events)

칸	값	원본과 비교
event_id	a7868199-2bc4-51ce-a0d6-3e58d973111d	**새로 생김.** 원본에 없어서 시각 · 유저 · 종류 · 상품 네 개를 해시해 만듦
event_type	transaction	event → 이름만 바꿈
event_time	2015-06-10 00:00:33.399 (UTC)	**1433894433399 밀리초 숫자 → 시각 타입.** 사람이 읽을 수 있고 날짜 · 분 단위 계산 가능
visitor_id	569031	visitorid → 이름 정리, bigint
item_id	417927	itemid → 이름 정리, bigint
transaction_id	1189	구매가 아니면 원본에선 빈칸, 저장 후엔 NULL
produced_at	2026-10-29 05:02:20	**새로 생김.** 생성기가 Kafka 로 보낸 시각
ingested_at	(DB 에만 있음)	**새로 생김.** DB 에 들어온 시각
schema_version / source	1 / replay-generator	**새로 생김.** 메시지 형식 버전, 출처
dt	2015-06-10	**새로 생김.** Parquet 날짜 폴더 기준

뭘 했나	어떻게	이유
자료형	"2015-05-03T03:00:04Z" 같은 글자를 시각으로, "693516" 을 숫자로 바꿈	이 숫자로 바꿔야 정렬이 편하기때문
빠진 값·이상한 값	필수 값이 없거나 종류가 view/addtocart/transaction 이 아니면 DLQ 에 넣는다.	
중복	event_id 가 같으면 제거했다.	

3. 최종 스키마

필드	자료형	뜻
event_id (고유 키)	uuid	(시각 | 유저 | 종류 | 상품) 을 해시한 값. 같은 내용이면 같은 값 → 이걸로 중복 판단
event_type	text	view / addtocart / transaction
event_time	timestamptz	클릭한 시각 (UTC). 집계 기준
visitor_id	bigint	유저
item_id	bigint	상품
transaction_id	bigint, null	구매일 때만 주문 번호
produced_at	timestamptz	생성기가 보낸 시각
ingested_at	timestamptz	DB 에 들어온 시각
schema_version	int	메시지 형식 버전
source	text	출처 (replay-generator)

4. 저장 포맷과 — 조회용 PostgreSQL, 보관용 Parquet

어디에	실제로 저장된 것	왜 여기에
PostgreSQL	최근 한달간의 데이터 및 사용자가 조회 하고싶은 데이터	모든 데이터를 넣고 조회하기에는 무겁기때문에 조회에 딱 필요한 정도만 postgres에 올림.
Parquet	모든 데이터	Parquet 은 용량이 적기 때문에 모든 데이터를 parquet으로 보관한다.
DLQ (Kafka 토픽)	깨진 메시지 2건 (JSON 아님 · 종류 이상)	오류 메시지는 다른 토픽에 저장.